

- 现在我已经掌握了所有信息。以下是全面分析：

一、SOC侧（APP）静态库依赖图

1. libfawhq_e001_10_com.so（通信域）

链接命令（consys_bf.cmake:213）：

```
--whole-archive
├─ plat_bf_cdd          (SDK: msg_route_soc, sig_route_soc, service_route, x_dom_can_rt_soc, x_dom_someip_rt, dcms_adapt, ...)
├─ dcms_dcms           (DCMS IPC 传输层库)
├─ APP_CORE_LIB        (app_core 业务逻辑)
├─ oem_feature         (OEM 功能)
├─ x_dom_can_rt_gen_lib_com (CAN comif 自动生成代码 - COM 侧)
├─ x_dom_someip_rt_gen_lib_com (SOME/IP comif 自动生成代码 - COM 侧)
├─ autosar_com         (AUTOSAR BSW: SomeIpTp/Sd/LdCom/Com/PduR/SoAd/...)
├─ oem_proxy           (OEM 代理)
--no-whole-archive
```

关键点：autosar_com 包含了替换过的 Cdd_SomeIpTp.c（原 SIP 的 Cdd_SomeIpTp.c 被移除）。编译时如果 CONFIG_X_DOM_SOMEIP_RT_COM=ON，则定义 -DX_DOM_SOMEIP_RT_COM_EN，激活其中的 comif 代码路径。

2. libuds_v4.so（UDS诊断域）

链接命令（consys_bf.cmake:237）：

```
--whole-archive
├─ plat_bf_cdd          (SDK: service_route, x_dom_someip_rt, ...)
├─ dcms_dcms           (DCMS IPC 传输层库)
├─ UDSONIP_TARGET      (UDS over IP 协议栈)
├─ diagproxy_ge        (诊断代理)
├─ x_dom_someip_rt_gen_lib_uds_v4 (SOME/IP comif 自动生成代码 - UDS 侧)
├─ autosar_uds_v4      (AUTOSAR BSW: Dcm/DoIP/PduR/SoAd/..., 不含LdCom/SomeIpTp/Sd)
--no-whole-archive
PRIVATE: DCOS_LIBRARIES, mini_dcos
```

关键点：autosar_uds_v4 不包含 LdCom、Sd、SomeIpTp —— 这些是 COM 域的。UDS 域有自己的 AUTOSAR 堆栈（Dcm、DoIP、PduR、SoAd），用于 danvince 诊断路径。

3. libfawhq_e001_10_app.so（AD/应用域）

来自 consys_ad.cmake，类似模式：

```
--whole-archive
├─ plat_bf_cdd + plat_ad_app_core + vehicle_proxy + adapter + fusa
├─ x_dom_can_rt_gen_lib_app (条件)
├─ x_dom_someip_rt_gen_lib_app (条件)
├─ app_业务库 (domain_function, gateway, ia, ...)
--no-whole-archive
```

4. 编译时依赖关系（UDS v4 不链接 COM 的 .so）

UDS v4 在编译时依赖 COM 生成的 comif 头文件（例如 x_dom_someip_rt_gen_com/x_dom_someip_rt_cfg.h），但不链接 libfawhq_e001_10_com.so。它是独立进程，通过 DCMS IPC 在运行时通信。

二、FW侧（MCU）依赖

FW 侧使用 Makefile 构建（非 CMake），直接将目标文件链接为固件 ELF/AXF/BIN 镜像。

目录结构（Makefile2）：

```
dsar_fw/
├─ app_core/          (appl_main, diag, dmm, fusa, bsw_manager_v2, can_com_platform, can_gateway, secoc, rtmon, ...)
├─ ctrl/              (控制算法: math library, vehicle control)
├─ proxy_comm/        (通信代理: comm, proxy_sensor, recorder, blf_recorder)
├─ dsar-bsw/          (BSW 基础软件层 - 可能来自 conan)
├─ product/           (产品特定配置、链接脚本)
├─ xwire/             (跨线通信: vehicle_info, sim, oemdata_ctrl, ...)
```

MCU 平台：SA8650 R52 核心（sa86x0_r52_config.mk）。

MCU 通过 CAN 总线与外部 ECU 通信，将 CAN 帧通过 DCMS 主题 "/x_dom_can_rt/mcu_tx_rx_msgs" 转发给 SOC，并从 SOC 接收写入命令（"/x_dom_can_rt/write_to_mcu_msgs"）。

三、三条消息路径

路径1：danvince（AUTOSAR 进程内）路径

这是旧架构 —— 数据通过标准 AUTOSAR BSW 堆栈在同一进程内从网络到应用层流动。

A. COM 域内的 SOME/IP danvince 路径

SOME/IP 网络数据包

- SoAd (Socket Adapter)
- SomeIpTp (SOME/IP Transport Protocol)
- LdCom (Large Data COM)
- PduR (PDU Router)
- Com (Communication)
- 应用层 SWC (进程内直接消费)

在 `Cdd_SomeIpTp.c` 中, 没有 `#ifdef X_DOM_SOMEIP_RT_COM_EN` 保护的 service 走这条路 —— 它们只打印 `SOMEIP_DEBUG_INFO`, 数据经由 `LdCom → PduR → Com` 传递给同一 `libfawhq_e001_10_com.so` 进程内的应用层代码。

B. UDS 域内的诊断 danvince 路径

DoIP 以太网诊断请求

- SoAd (Socket Adapter)
- DoIP (Diagnostic over IP)
- PduR (PDU Router) — 路由至 Dcm
- Dcm (Diagnostic Communication Manager) — 处理 UDS 服务
- Dcm_Callout_Stubs.c — 认证、安全访问等回调
- DiagnosticService.c — 具体诊断服务实现 (0x22/0x2E/0x31/0x34/0x36/0x37等)
- Rte_Dcm.h — RTE 接口
- DiagSwc_Callback.c / DiagApp.c — 诊断应用层回调

所有操作都在 `libuds_v4.so` 进程内。关键文件:

- `Dcm_Callout_Stubs.c` — 认证/安全访问
- `DiagnosticService.c` — 每个 UDS-SID 的服务处理程序
- `dji_rte.c` — RTE 调度粘合代码

对于 `fawhq_e001_10`, CAN 信号的 danvince 路径未使用 (`CONFIG_X_DOM_CAN_RT_UDS_V4=OFF`), 因为 CAN 通信通过 `comif` 经由 COM 域。

路径2: DCMS (直接 IPC 传输) 路径

DCMS (DJI Communication Middleware Service) 是底层传输层 —— 一个发布/订阅 + 客户端/服务器 IPC 机制, 被所有跨域/跨核通信使用。它是传输层, 而非架构本身。

CAN 帧: MCU → SOC (原始 DCMS, 无 `comif` 路由)

MCU (R52)	SOC (COM 域)
CAN 总线 → CAN 驱动	DCMS 传输
→ dcms 主题: "/x_dom_can_rt/mcu_tx_rx_msgs"	→ <code>msg_route_mcu_tx_rx_msgs_topic_cbk()</code>
	→ <code>mcu_can_msg_queue.push()</code>
	→ <code>msg_route_mcu_tx_rx_can_msgs_proc()</code>
	→ 根据 (channel, can_id) 查找 <code>CanMsg</code>
	→ <code>CanMsg::set_data()</code> — 缓存
	→ <code>msg_route_can_msg_cb_ctx->set_recv_msg()</code>
	→ <code>CanMsgCbItem::call()</code> — 用户回调
	→ <code>sig_route_send_mcu_tx_rx_sigs()</code>
	→ 将缓存的 CAN 信号转发给 UDS/AD

SOC (COM 域)	MCU (R52)
<code>msg_route_write_msgs_to_mcu_proc()</code>	
→ 遍历 <code>msg_route_write_to_mcu_msg_id_pairs_map</code>	
→ 从 <code>CanMsg</code> 缓存读取信号 → 填充 <code>can_msg_t</code>	
→ dcms 主题: "/x_dom_can_rt/write_to_mcu_msgs"	→ MCU 接收
	→ CAN 驱动 → 发送到总线

文件位置: `msg_route_soc.cpp:489-511` (接收)、`msg_route_soc.cpp:241-329` (发送)。

CAN 信号: COM → UDS/AD (通过 `comif` 信号转发, 但使用原始 DCMS 主题)

COM: <code>sig_route_send_mcu_tx_rx_sigs_proc()</code>	UDS/AD: <code>sig_route_mcu_rx_sigs_topic_cbk()</code>
→ <code>sig_route_send_mcu_tx_rx_sigs()</code> [自动生成]	→ <code>sig_route_update_mcu_tx_rx_sigs_from_com()</code>
→ 将所有信号打包为二进制	→ 反序列化二进制 → 更新本地缓存
→ <code>dcms_mcu_topic_send("/mcu_tx_rx_sigs")</code>	

文件位置: `sig_route_soc.cpp:26-31` (COM 侧)、`sig_route_soc.cpp:68-72` (UDS/AD 侧)。

路径3: `comif` (基于 DCMS 的 JSON 配置路由) 路径

这是新架构 —— 一个 JSON 配置驱动的跨域服务路由框架, 构建于 DCMS 之上。它添加了路由表、服务仲裁、序列化/反序列化以及自动生成的 `SigIf` 接口。

SOME/IP 服务: 外部 ECU → COM → UDS/AD

步骤1 — COM 接收 SOME/IP 数据 (来自外部 ECU):

外部 ECU 发送 SOME/IP 服务数据

- SoAd → SomeIpTp → Cdd_SomeIpTp.c 回调
- #ifdef X_DOM_SOMEIP_RT_COM_EN (125个服务)
- x_dom_someip_rt_rx_handle(server_id, data, len) - 【comif 入口】

这在 Cdd_SomeIpTp.c:427-428 中:

```
#ifdef X_DOM_SOMEIP_RT_COM_EN
    x_dom_someip_rt_rx_handle(X_DOM_SOMEIP_RT_SERVER_ID..., Data + OFFSET, Len);
#endif
```

步骤2 — COM comif 路由引擎:

```
x_dom_someip_rt_rx_handle() [x_dom_someip_rt.cpp:87]
- svc_route_rx_handle(id, data, len) [service_route.cpp:80]
- 根据 id 在 svc_route_rcv_svc_info_map_ptr 中查找 SvcItemInfo
- item_ptr->set_origin_data(id, data, len) - 缓存数据
- 如果是即时服务:
-   - svc_route_immediately_pub() [service_route.cpp:545]
-   - 遍历 svc_route_rcv_svc_info_list_ptr
-   - 对每个目标节点:
-     - check_and_send_svc_via_dcms(id, data, len, topic)
```

步骤3 — DCMS 主题转发 (COM → UDS/AD):

```
COM 服务路由 (每50ms) DCMS 传输
-----
svc_route_cycle_pub() 主题: "/x_dom_someip_rt/com_to_app/uds_app"
- check_and_send_svc_list_via_dcms_on_timeout() 主题: "/x_dom_someip_rt/com_to_app/ad_app"
- SvcDataPackHdl::append_item() - 将多个服务打包
- SvcDataPackHdl::get_data() - 获取打包后的二进制
- dcms_mcu_topic_send(topic, packed_data, len) ----->
```

文件位置: [service_route.cpp:561-573](#)。

步骤4 — UDS/AD 接收:

```
svc_route_com_to_app_topic_cbk(data, len) [service_route.cpp:147]
- SvcDataUnPackHdl::get_next_data() - 逐个解包
- 根据 id 在 svc_route_rcv_svc_info_map_ptr 中查找 SvcItemInfo
- item_ptr->set_com_rcv_count() / set_origin_data() - 更新缓存
- SigIf_OnRcvData(id) - 触发自动生成的信号接口回调 [x_dom_someip_rt_sigif_get.cpp]
- switch(service_id) - 反序列化 - 应用回调
```

反向路径: UDS/AD → COM (应用发送数据至外部 ECU):

```
UDS/AD 应用调用 SigIf 设置/更新函数
- SvcItemInfo::set_usr_data<template>()
- 每50ms: svc_route_main_function() (app_service_enabled 分支)
- send_svc_list_via_dcms_on_timeout(map, "/x_dom_someip_rt/app_to_com")
- SvcDataPackHdl 打包 - dcms_mcu_topic_send()
```

```
COM 接收:
svc_route_app_to_com_topic_cbk() [service_route.cpp:107]
- SvcDataUnPackHdl::get_next_data()
- item_ptr->fwd_svc_via_someip() - 转发至 AUTOSAR SOME/IP 堆栈
- SOME/IP → 外部 ECU
```

信号仲裁 (多源信号冲突解决): [service_route.cpp:192-343](#)

多个来源 (SOC, MCU_SAIL, MCU_VIP) 发送同一信号

- svc_sig_arb_app_to_com_topic_cbk()
- 基于优先级仲裁 + 超时回退 + 链路延迟/丢包检测
- svc_route_update_service_data_ptr() 更新获胜者数据

四、架构总结对比

维度	danvince (旧)	DCMS (传输层)	comif (新)
数据如何流动	AUTOSAR BSW 堆栈进程内	原始主题 send/callback	配置驱动的服务路由
跨域?	否 — 同一进程	是 — IPC	是 — 在 DCMS 之上
配置方式	DaVinci Configurator 生成的 C 文件	CMake 主题列表	JSON → 自动生成的 C++ 路由表
COM 侧入口	LdCom / SomeIpTp 直接至 Com	msg_route_mcu_tx_rx_msgs_topic_cbk	x_dom_someip_rt_rx_handle()
UDS 侧入口	Dcm / DdIP 进程内	sig_route_mcu_rx_sigs_topic_cbk	svc_route_com_to_app_topic_cbk()
序列化	AUTOSAR 标准 (SOME/IP 线格式)	原始 C 结构体二进制	SvcDataPackHdl / SvcDataUnPackHdl 带 ID 的二进制
服务发现	AUTOSAR Sd (FindService/OfferService)	无 (点对点主题)	自动生成的路由表 svc_route_rcv_svc_info_map
	Cdd_SomeIpTp.c 无 comif 保护的 Service LdCom		service_route.cpp x_dom_someip_rt.cpp

关键文件 (COM)	Cdd_SomeipRt.c, DiagnosticService.c, Dcm_Callout_Stubs.c, dji_rte.c, Com	msg_route_soc.cpp, sig_route_soc.cpp	service_route.cpp, x_dom_someip_rtcp, x_dom_someip_rt_gen_com/
关键文件 (UDS)	DiagnosticService.c, Dcm_Callout_Stubs.c, dji_rte.c	sig_route_soc.cpp (XDOMCANRT_DJI_COM_SERVICE_EN=0)	service_route.cpp, x_dom_someip_rt.cpp, x_dom_someip_rt_gen_uds/

关于遗留重叠：如你所指出的，Cdd_SomeipRt.c 同时包含两个路径——125 个服务走 comif (#ifdef X_DOM_SOMEIP_RT_COM_EN)，其余走 danvince (仅 SOMEIP_DEBUG_INFO)。这是生成代码中可预期的遗留模式：已迁移的服务获得 comif 转发，其他服务保留在 AUTOSAR 进程内路径。

FW侧没有comif接口吗，另外我不理解的是com域.so依赖RT_COM静态库，uds域走comif接口也会依赖RT_COM静态库，两个so文件会编译同一份静态库吗，相同的代码会生成两次吗，如果运行在静态库中的线程也会跑两份吗？

Thought for 9s >

Bash

List SDK cdd directory contents

```
IN ls /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/x_dom_can_rt/ /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/x_dom_someip_rt/:
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/x_dom_can_rt/:
CMakeLists.txt
msg_route_soc.cpp
```

Bash

Search for comif references in FW/MCU code

```
IN grep -rn "x_dom_can_rt|x_dom_someip_rt|comif" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/ --include="*.c" --include="*.cpp" --include="*.h" --
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/include/common/include_all.mk
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/build/main_rh850.mk
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/app_core/common/include_asw.mk
```

Read

x_dom_can_rt_internal.h

Thought for 6s >

Read

CMakeLists.txt

Read

CMakeLists.txt

Read

x_dom_can_rt_cfg.c

Read

x_dom_can_rt_sigif.c

Thought for 7s >

Bash

Check how cdd CMakeLists includes x_dom_can_rt and x_dom_someip_rt

```
IN grep -rn "x_dom_can_rt|x_dom_someip_rt" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/CMakeLists.txt 2>/dev/null
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/CMakeLists.txt:126:add_subdirectory(x_dom_can_rt)
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/CMakeLists.txt:27:add_subdirectory(x_dom_someip_rt)
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq-plat/dsar-plat-bf/dsar_app/cdd/CMakeLists.txt:67: x_dom_someip_rt
```

Read

x_dom_can_rt_cfg.c

Read

comif.c

Thought for 9s >

Read

CMakeLists.txt

Read

x_dom_can_rt_cfg.h

Thought for 35s >

Read

x_dom_can_rt_sigif.c

Bash

Find where x_dom_can_rt SDK sources are compiled into static libs

```
IN grep -rn "x_dom_can_rt_sdk_src_list\|X_DOM_CAN_RT_SRC\|msg_route_soc.cpp\|sig_route_soc.cpp\|x_dom_can_rt_soc.cpp" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-h
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/proxy/x_dom_can_rt_gen_uds/CMakeLists.txt:3:set(x_dom_can_rt_inc_list ${CM
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/proxy/x_dom_can_rt_gen_uds/CMakeLists.txt:8:set(x_dom_can_rt_sdk_src_list
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/fawhq_e001_10/proxy/x_dom_can_rt_gen_uds/CMakeLists.txt:9: ${X_DOM_CAN_RT_SRC}/si
```

Thought for 13s >

好问题，逐一回答。

1. FW侧有没有comif？

有，FW侧也有comif，但形式不同。

fawhq_e001_10/x_dom_can_rt_config/目录下有自动生成的comif代码：

- x_dom_can_rt_cfg.c — 36个CAN消息过滤器（与SOC侧同源JSON生成）
- x_dom_can_rt_sigif.c — 信号读写接口
- x_dom_can_rt_cfg.h — 功能开关宏

关键区别在头文件引用：

	SOC侧	FW/MCU侧
SDK头文件	x_dom_can_rt_soc.h	x_dom_can_rt_mcu.h
信号处理	sig_handle_soc.hpp	sig_handle_mcu.h
消息路由	msg_route_soc.h	msg_route_mcu.h
信号发送	DCMS dcms_mcu_topic_send()	AUTOSAR Com_SendSignal()

FW侧的SigIf函数直接调用AUTOSAR Com栈发送CAN信号（x_dom_can_rt_sigif.c:30）：

```
retval = sig_if_set_raw_value(...);  
// 通过 AUTOSAR Com_SendSignal 发送到 CAN 总线，不需要 DCMS 中转
```

SOC侧则通过DCMS主题转发。这是因为MCU直接挂在CAN总线上，无需IPC。

2. 两个.so会编译同一份静态库吗？代码会生成两次吗？

是的，但分情况讨论。

x_dom_can_rt SDK (msg_route_soc.cpp / sig_route_soc.cpp / x_dom_can_rt_soc.cpp)

这些是源文件直接输入每个静态库的。看两个CMakeLists.txt的区别：

COM侧 x_dom_can_rt_gen_com/CMakeLists.txt:7:

```
aux_source_directory(${X_DOM_CAN_RT_SRC} x_dom_can_rt_sdk_src_list)  
# 收集 X_DOM_CAN_RT_SRC 目录下所有 .cpp -> msg_route_soc, sig_route_soc, x_dom_can_rt_soc, p2p_parser 全部编入  
add_library(x_dom_can_rt_gen_lib_com ${PRODUCT_NAME} STATIC  
    ${x_dom_can_rt_gen_src_list} ${x_dom_can_rt_sdk_src_list})
```

UDS侧 x_dom_can_rt_gen_uds/CMakeLists.txt:8-12:

```
set(x_dom_can_rt_sdk_src_list  
    ${X_DOM_CAN_RT_SRC}/sig_route_soc.cpp  
    ${X_DOM_CAN_RT_SRC}/x_dom_can_rt_soc.cpp)  
# 只编 sig_route_soc + x_dom_can_rt_soc, 不编 msg_route_soc (UDS不直接收CAN帧)
```

结论：同一个源文件 sig_route_soc.cpp 被编译了两次——一次带着COM侧的 x_dom_can_rt_cfg.h（定义XDOMCANRT_DJI_COM_SERVICE_EN=1），一次带着UDS侧的 x_dom_can_rt_cfg.h（定义XDOMCANRT_DJI_COM_SERVICE_EN=0）。生成两份不同的.o，分别链接进 libfawhq_e001_10_com.so 和 libuds_v4.so。

x_dom_someip_rt SDK (service_route.cpp / x_dom_someip_rt.cpp)

这些不同。plat_bf_cdd的cdd/CMakeLists.txt:67把 x_dom_someip_rt 作为OBJECT库链接进**plat_bf_cdd.so（共享库）**。两个.so都动态链接 plat_bf_cdd.so，共享同一份代码，不重复。

最终.so包含情况

```
libfawhq_e001_10_com.so (COM进程加载)  
├─ [静态链接] sig_route_soc.o (XDOMCANRT_DJI_COM_SERVICE_EN=1)  
├─ [静态链接] x_dom_can_rt_soc.o (domain="com")  
├─ [静态链接] msg_route_soc.o (COM特有，负责接收MCU CAN帧)  
├─ [静态链接] x_dom_can_rt_cfg_com.o (自动生成：37个CanMsg，~60个SigHdl)  
├─ [静态链接] x_dom_someip_rt_cfg_com.o (自动生成：路由表)  
└─ [动态链接] -> plat_bf_cdd.so (含 service_route.o, x_dom_someip_rt.o)
```

```
libuds_v4.so (UDS进程加载)  
├─ [静态链接] sig_route_soc.o (XDOMCANRT_DJI_COM_SERVICE_EN=0)  
├─ [静态链接] x_dom_can_rt_soc.o (domain="doipv4")  
├─ [静态链接] x_dom_someip_rt_cfg_uds.o (自动生成：路由表+74个服务的回调)  
└─ [动态链接] -> plat_bf_cdd.so (含 service_route.o, x_dom_someip_rt.o)
```

3. 线程也会跑两份吗？

是的，关键在x_dom_can_rt_soc.cpp:82-127的 x_dom_can_rt_init()：

```
int32_t x_dom_can_rt_init(const char* name)  
{  
    if (strcmp(name, "com") == 0) // COM进程调用  
    {  
        task_name = "com_app_comif";  
        // 注册 COM主题: mcu_tx_rx_msgs, write_to_mcu_msgs, write_sigs, read_sigs_app...  
    }  
}
```

```
else if (strcmp(name,"doipv4") == 0) // UDS进程调用
{
    task_name = "uds_app_comif";
    // 注册 UDS主题: write_sigs(client), read_sigs_uds(server) - 仅2个主题
}

// 每个进程都创建自己的50ms周期任务
add_period_work_with_node_name(task_name.c_str(),
    (task_name + "_can").c_str(), task_mid, APP_PERIOD_50MS,
    x_dom_can_rt_main_func, nullptr); // - 每个进程独立运行
}
```

x_dom_can_rt_main_func(), msg_route_main_function() + sig_route_main_function() — 每个进程的50ms任务独立运行，通过编译时宏（不同产品的 x_dom_can_rt_cfg.h）和运行时参数（"com" vs "doipv4"）决定行为：

	COM进程的线程	UDS进程的线程
msg_route_main_function	处理MCU CAN帧、发送写入消息	不执行 (XDOMCANRT_MSG_ROUTE_EN=0)
sig_route_send_mcu_tx_rx_sigs	发送信号给UDS/AD	不执行
msg_route_main_function	处理MCU CAN帧、发送写入消息	不执行 (XDOMCANRT_MSG_ROUTE_EN=0)
sig_route_send_mcu_tx_rx_sigs	发送信号给UDS/AD	不执行
sig_route_send_write_to_mcu_sigs	不执行	发送写入信号给COM
sig_route_recv_write_to_mcu_sigs_topic_cbk	接收UDS/AD的写入信号	不执行
sig_route_mcu_rx_sigs_topic_cbk	不执行	接收COM转发的CAN信号

所以确实是两个独立进程、两份代码、两个独立线程，各自扮演不同角色。COM负责与MCU通信、管理CAN帧缓存、转发给UDS/AD；UDS只负责接收COM转发的信号、发送写入请求回COM。